Effects in parallel, with concurrent monads

Tarmo Uustalu
Reykjvik University / Tallinn University of Technology
Lambda World '26, Torremolinos, 29–30 Oct. 2026

What this is about

In functional programming, monads are a structure facilitating sequential composition of effectful functions

Is there a similar thing for parallel composition?

Yes, but underappreciated as such: applicatives [lax monoidal functors]

But for both of the two at once?

Yes, and you deserve to know: concurrent monads

Monads in Haskell

a bit differently than in Prelude

```
class Functor t => Mnd t where
  return :: x -> t x
  join :: t (t x) -> t x

newtype Reader s x = R (s -> x)

instance Mnd (Reader s) where
  return x = R (\ _ -> x)
  join (R f) = R (\ s -> let R g = f s in g s)

newtype State s x = S (s -> (s, x))

instance Mnd (State s) where
  return x = S (\ s -> (s, x))
  join (S f) = S (\ s -> let (s', S g) = f s in g s')
```

The mission of a monad

A monad provides

- a combinator for sequential composition of functions with some kind of effect
- guarantees that it is sensible:
  - with the right type
  - with the right equational property[1] (= associativity)

[1] if the monad is a monad indeed

Sequential composition, with the right type

T – a functor, for some kind of effect

Effectful functions $X \to Y$ are the same as pure functions $X \to TY$

$$\begin{array}{c} k : X \longrightarrow \boxed{TY} \quad \ell : \boxed{Y} \longrightarrow TZ \\ \vdots\ ? \\ X \longrightarrow TZ \end{array}$$

$$\begin{array}{c} \dfrac{\ell : Y \longrightarrow TZ}{T\ell : \boxed{TY} \longrightarrow \boxed{T(TZ)}} \\ \vdots\ ? \\ X \longrightarrow \boxed{TZ} \end{array} \qquad k : X \longrightarrow \boxed{TY}$$

$$\dfrac{k : X \longrightarrow \boxed{TY} \qquad \dfrac{\dfrac{\ell : Y \longrightarrow TZ}{T\ell : TY \longrightarrow T(TZ)} \quad \mu_Z : T(TZ) \longrightarrow TZ}{T\ell \triangleright \mu_Z : \boxed{TY} \longrightarrow \boxed{TZ}}}{k \blacktriangleright \ell := k \triangleright T\ell \triangleright \mu_Z : X \longrightarrow TZ}$$

$$\mathsf{id}^{\blacktriangleright}_X := \eta_X : X \longrightarrow TX$$

Sequential computation, with the right equations

$$\frac{\mu_{TX} \triangleright \mu_X = T\mu_X \triangleright \mu_X}{\begin{array}{c}\vdots\\ (k \blacktriangleright \ell) \blacktriangleright m = k \blacktriangleright (\ell \blacktriangleright m)\end{array}}$$

$$\frac{\eta_{TX} \triangleright \mu_X = \mathsf{id}_{TX}}{\begin{array}{c}\vdots\\ \mathsf{id}^{\blacktriangleright}_X \blacktriangleright k = k\end{array}}$$

$$\frac{\mathsf{id}_{TX} = T\eta_X \triangleright \mu_X}{\begin{array}{c}\vdots\\ k = k \blacktriangleright \mathsf{id}^{\blacktriangleright}_X\end{array}}$$

A concurrent monad's mission

A concurrent monad provides

- combinators for sequential and parallel composition of functions with some kind of effect
- guarantees that they are sensible:
  - with the right typing
  - with the right (in)equational properties [2]
    (= both monotone, both associative, interrelating via inequational interchange)

[2] if the concurrent monad is a concurrent monad indeed

Parallel composition, with the right typing

$$\dfrac{\dfrac{k : X \longrightarrow TY \quad \ell : U \longrightarrow TV}{k \times \ell : X \times U \longrightarrow \boxed{TY \times TV}} \quad \vdots\ ? \quad \phi_{Y,V} : TY \times TV \longrightarrow T(Y \times V)}{k \parallel \ell := k \times \ell \triangleright \phi_{Y,V} : X \times U \longrightarrow T(Y \times V)}$$

$$X \times U \longrightarrow \boxed{T(Y \times V)}$$

$$\mathsf{jd} := \phi^0 : 1 \longrightarrow T1$$

Concurrent monads in Haskell, reader, state

```
class Functor t => Lmf t where
  nil :: () -> t ()
  merge :: (t x, t y) -> t (x, y)

class (Monad t, Lmf t) => ConcurMnd t where

newtype Reader s x = R (s -> x)

instance Lmf (Reader s) where
  nil () = R (\ _ -> ())
  merge (R f, R g) = R (\ s -> (f s, g s))

instance ConcurMnd (Reader s)

instance MeetSemi s => Lmf (State s) where
  nil () = S (\ _ -> (top, ()))
  merge (S f, S g) = S (\ s -> let ((s0, x), (s1, y)) = (f s, g s) in (meet (s0, s1), (x, y)))

instance MeetSemi s => ConcurMnd (State s)
```

A concurrent monad of resumptions for shared state and interleaving

```
data Res s x = Ret x | Grab (s -> [(s, Res s x)])

instance Mnd (Res s) where
  return x = Ret x
  join (Ret r)  = r
  join (Grab k) = Grab (\ s -> [ (s', join r) | (s', r) <- k s ])

instance Lmf (Res s) where
  nil () = Ret ()
  merge (Ret x, Ret y)  = Ret (x, y)
  merge (Ret x, Grab l) =
    Grab (\ s -> [ (s', merge (Ret x, r)  | (s', r) <- l s ])
  merge (Grab k, Ret y) =
    Grab (\ s -> [ (s', merge (r, Ret y)  | (s', r) <- k s ])
  merge (Grab k, Grab l) =
    Grab (\ s -> [ (s', merge (r, Grab l) | (s', r) <- k s ]
              ++ [ (s', merge (Grab k, r) | (s', r) <- l s ])
```

We saw

Parallel composition is as fundamental as sequential composition, also for effectful functions

Monads axiomatize only sequential composition

Concurrent monads axiomatize both compositions

The two compositions interrelate via inequational interchange

This presupposes an order on functions between the same types

We also learned

Concurrent monads relax commutative monads,
which have equational interchange

Concurrent monads differ from strong monads,
which support parallel composition only of an effectful function and a pure function,
with equational interchange

We did not see

Concurrent monads are concurrent monoids in the right place
(a suitable ordered duoidal category)

Concurrent monads are (2-categorical) monads in the right place
(a suitable 2-category)

Concurrent monads are meaningfully definable also for orders between effectful functions only

For more information

E Rivas, M Jaskelioff, preprint 2019

H Paquet, P Saville, LICS '24

E Rivas, T Uustalu, PPDP '24

F Breuvart, H Paquet, FSCD '25

B Gunnarsson, U of Iceland MSc thesis '26